



Creación e Implementación de un Data Warehouse usando PostgreSQL y Jupyter con Python

Extracción de Conocimiento en
Bases de Datos

Ingeniería en Desarrollo y Gestión de Software

Universidad Tecnológica Del Centro De Veracruz

Alumno:

Nombre y matrícula:

Arenas López Alejandro-202331001102

Profesor:

MRYSI. MARIA REINA ZARATE NAVA

Extracción de Conocimiento en Bases de Datos

CUITLÁHUAC, VER.

JUN, 2026



Índice/Contenido

Investigación Documental	4
¿Qué es un Data Warehouse?	4
Diferencias entre base de datos operacional y Data Warehouse	4
Ventajas del uso de un Data Warehouse.....	5
Arquitecturas comunes (Esquemas de Modelado Dimensional)	5
Instalación y Configuración.....	7
Instalación de PostgreSQL.....	7
Creación de la base de datos del Data Warehouse.....	8
Creación de tablas de dimensión y tablas de hechos	8
Carga de Datos.....	9
Análisis y Visualización.....	12
Conclusiones	16

Investigación Documental

¿Qué es un Data Warehouse?

Un Data Warehouse (Almacén de Datos) es un sistema centralizado de gestión de datos diseñado específicamente para habilitar y respaldar las actividades de Inteligencia de Negocios (BI) y la toma de decisiones empresariales. Funciona como un gran repositorio donde se integra, depura y almacena información histórica y actual proveniente de una o múltiples fuentes heterogéneas (como bases de datos transaccionales, CRMs, ERPs, archivos planos, etc.).

Según Bill Inmon, considerado el padre del Data Warehousing, un almacén de datos tiene cuatro características fundamentales:

- **Orientado a temas:** Se organiza alrededor de los temas principales de la empresa (ventas, clientes, productos) en lugar de las aplicaciones transaccionales.
- **Integrado:** Unifica datos de diferentes fuentes bajo un formato y estructura consistentes.
- **Variante en el tiempo:** Mantiene un registro histórico de los datos, permitiendo el análisis de tendencias a lo largo del tiempo.
- **No volátil:** Una vez que los datos entran al almacén, no se modifican ni se eliminan; son de solo lectura para fines analíticos.

Diferencias entre base de datos operacional y Data Warehouse

Las bases de datos operacionales, conocidas como sistemas **OLTP** (Online Transaction Processing), están diseñadas para el día a día. Por el contrario, los Data Warehouses son sistemas **OLAP** (Online Analytical Processing), diseñados para el análisis.

Característica	Base de Datos Operacional (OLTP)	Data Warehouse (OLAP)
Propósito principal	Soportar las operaciones diarias y el procesamiento de transacciones.	Soportar el análisis de datos, reportes y toma de decisiones estratégicas.
Tipo de datos	Datos actuales, altamente detallados y dinámicos.	Datos históricos, resumidos y consolidados.

Vigencia: 2020.09.17-2024.09.17
Registro: RPI/L: 096

Dirección
Av. Universidad No. 350, Carretera Federal Cuitláhuac- La Tinaja
Congregación Dos Caminos, C.P. 94910 Cuitláhuac, Veracruz
Tel. (278) 73 2 20 50
www.utcv.edu.mx

Diseño y Estructura	Altamente normalizada (para evitar redundancia y agilizar inserciones).	Desnormalizada (esquemas de estrella/copo de nieve) para agilizar lecturas.
Volumen de datos	Generalmente en el rango de Gigabytes (GB).	Masivo, abarcando Terabytes (TB) o Petabytes (PB).
Operaciones típicas	Lectura, escritura, actualización y borrado continuos y rápidos (CRUD).	Principalmente consultas complejas de lectura (Select); actualizaciones periódicas en bloque mediante procesos ETL.
Usuarios principales	Empleados operativos, sistemas automatizados, clientes.	Analistas de datos, gerentes, directivos (Data Scientists).

Ventajas del uso de un Data Warehouse

Implementar un almacén de datos ofrece múltiples beneficios estratégicos y técnicos para una organización:

- **Única fuente de verdad:** Consolida los datos de toda la empresa, garantizando que todos los departamentos basen sus análisis y decisiones en la misma información, eliminando discrepancias.
- **Mejora en la toma de decisiones:** Al proporcionar acceso rápido a datos históricos e integrados, los líderes pueden identificar tendencias, analizar el comportamiento del mercado y tomar decisiones basadas en datos (Data-Driven).
- **Alto rendimiento analítico:** Al separar las consultas analíticas pesadas de las bases de datos operacionales, se evita que los sistemas del día a día (ventas, inventario) se ralenticen cuando se ejecutan reportes complejos.
- **Calidad y consistencia de datos:** Antes de ingresar al Data Warehouse, los datos pasan por un proceso de Extracción, Transformación y Carga (ETL) donde se limpian, estandarizan y corrigen errores.
- **Inteligencia Histórica:** Permite comparar el rendimiento actual de la empresa con años o meses anteriores de forma ágil, algo que es muy costoso a nivel computacional en sistemas transaccionales.

Arquitecturas comunes (Esquemas de Modelado Dimensional)

Para organizar los datos dentro de un Data Warehouse y optimizar las consultas analíticas, se utilizan modelos dimensionales. Los más comunes son:

Vigencia: 2020.09.17-2024.09.17
Registro: RPHL 096

A. Esquema en Estrella (Star Schema)

Es la arquitectura más simple y utilizada. Consiste en una gran Tabla de Hechos (Fact Table) en el centro, que contiene las métricas cuantitativas o datos transaccionales (ej. cantidad vendida, ingresos). Esta tabla central se conecta a múltiples Tablas de Dimensiones (Dimension Tables) dispuestas a su alrededor como las puntas de una estrella. Las dimensiones describen los atributos de los hechos (ej. tiempo, producto, cliente, tienda).

- *Ventaja:* Consultas muy rápidas y diseño fácil de entender por usuarios de negocio.
- *Desventaja:* Redundancia de datos en las dimensiones (desnormalización).

B. Esquema en Copo de Nieve (Snowflake Schema)

Es una variación del esquema en estrella. La diferencia radica en que las tablas de dimensiones están normalizadas (divididas en sub-dimensiones adicionales). Por ejemplo, la dimensión "Ubicación" podría dividirse en "Ciudad", y esta a su vez en "País". La representación visual se asemeja a un copo de nieve.

- *Ventaja:* Ahorra espacio de almacenamiento y reduce la redundancia de datos.
- *Desventaja:* Consultas más complejas y lentas debido a la necesidad de múltiples uniones (JOINS) entre tablas.

C. Constelación de Hechos (Fact Constellation o Galaxy Schema)

Este esquema se utiliza en sistemas a nivel empresarial muy grandes y complejos. En lugar de tener una sola tabla de hechos, cuenta con **múltiples tablas de hechos que comparten tablas de dimensiones**. Por ejemplo, una tabla de hechos para "Ventas" y otra para "Inventario" podrían compartir las dimensiones "Tiempo" y "Producto".

- *Ventaja:* Altamente flexible y permite cruzar información de diferentes áreas de la empresa.
- *Desventaja:* Diseño y mantenimiento arquitectónico muy complejo.

5. Herramientas y tecnologías para implementar un Data Warehouse

El panorama tecnológico actual ofrece soluciones variadas, destacando la fuerte tendencia hacia el Cloud Computing (la nube) debido a su escalabilidad.

Data Warehouses en la Nube (Cloud):

- **Google BigQuery:** Solución serverless, altamente escalable y con capacidades de Machine Learning integradas.

Vigencia: 2020.09.17-2024.09.17
Registro: RPIL: 096

- **Amazon Redshift:** Servicio de AWS ampliamente adoptado, diseñado para manejar petabytes de datos con alto rendimiento.
- **Snowflake:** Plataforma nativa de la nube conocida por su arquitectura única que separa el almacenamiento del poder de cómputo, permitiendo escalabilidad instantánea.
- **Microsoft Azure Synapse Analytics:** Evolución de SQL Data Warehouse, combina integración de datos, almacenamiento de datos empresariales y análisis de Big Data.

Data Warehouses Tradicionales / On-Premise:

- **Teradata:** Uno de los sistemas más robustos y clásicos para grandes corporativos.
- **Oracle Exadata:** Solución de hardware y software optimizada para cargas de trabajo de bases de datos Oracle.
- **Microsoft SQL Server:** Ampliamente utilizado en entornos corporativos tradicionales.

Tecnologías Complementarias (ETL/ELT): Para que un Data Warehouse funcione, requiere herramientas que muevan y transformen los datos desde el origen hasta el almacén. Destacan: Apache Airflow, dbt (data build tool), Talend, Informatica PowerCenter y AWS Glue.

Instalación y Configuración

Instalación de PostgreSQL

Para el despliegue del almacén de datos en un entorno Windows, se descargó el instalador interactivo oficial (proporcionado por EnterpriseDB). Durante la ejecución del asistente de instalación, se seleccionaron los componentes necesarios (PostgreSQL Server y pgAdmin), se asignó una contraseña segura para el usuario administrador por defecto (postgres) y se mantuvo el puerto de red predeterminado (5432). Una vez finalizada la instalación, se verificó que el servicio del servidor estuviera en ejecución a través de pgAdmin para asegurar la conexión.



Creación de la base de datos del Data Warehouse

Con el servidor activo, se procedió a crear la base de datos principal que funcionará como el repositorio central para la información analítica. El script SQL ejecutado fue el siguiente:

```
-- Creación de la base de datos principal
CREATE DATABASE data_warehouse_utcv;

-- Comando para conectarse a la base de datos (si se usa la terminal psql)
\c data_warehouse_utcv;
```

Así mismo nos ubicamos dentro de la base de datos creada.

EVIDENCIA

```
postgres=# CREATE DATABASE data_warehouse_utcv
postgres=# ;
CREATE DATABASE
postgres=# \c data_warehouse_utcv;
Ahora está conectado a la base de datos «data_warehouse_utcv» con el usuario «postgres».
data_warehouse_utcv=# _
```

Creación de tablas de dimensión y tablas de hechos

Para organizar la información dentro del Data Warehouse, se implementó un modelo de Esquema en Estrella (Star Schema). Este diseño consiste en separar el contexto del negocio en tablas de dimensión (como cliente, producto y tiempo) y centralizar las métricas transaccionales en una tabla de hechos (ventas) conectada a las dimensiones mediante llaves foráneas.

1. Tablas de Dimensión:

-- Dimensión Cliente

```
CREATE TABLE dim_cliente (
  id_cliente SERIAL PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  segmento VARCHAR(50),
  ciudad VARCHAR(50),
  pais VARCHAR(50)
);
```

-- Dimensión Producto

```
CREATE TABLE dim_producto (
  id_producto SERIAL PRIMARY KEY,
  nombre_producto VARCHAR(100) NOT NULL,
  categoria VARCHAR(50),
  precio_unitario DECIMAL(10,2)
);
```

-- Dimensión Tiempo (Crucial para análisis histórico)

```
CREATE TABLE dim_tiempo (
```



```
id_tiempo SERIAL PRIMARY KEY,  
fecha DATE NOT NULL,  
anio INT,  
mes INT,  
dia INT,  
trimestre INT  
);
```

2. Tabla de Hechos:

-- Tabla de Hechos:

```
Ventas CREATE TABLE fact_ventas (  
id_venta SERIAL PRIMARY KEY,  
id_cliente INT REFERENCES dim_cliente(id_cliente),  
id_producto INT REFERENCES dim_producto(id_producto),  
id_tiempo INT REFERENCES dim_tiempo(id_tiempo),  
cantidad INT NOT NULL,  
monto_total DECIMAL(12,2) NOT NULL  
);
```

EVIDENCIA

```
data_warehouse_utcv=# CREATE TABLE dim_cliente ( id_cliente SERIAL PRIMARY KEY, nombre VARCHAR  
(100) NOT NULL, segmento VARCHAR(50), ciudad VARCHAR(50), pais VARCHAR(50));  
CREATE TABLE  
data_warehouse_utcv=# CREATE TABLE dim_producto ( id_producto SERIAL PRIMARY KEY, nombre_produ  
cto VARCHAR(100) NOT NULL, categoria VARCHAR(50), precio_unitario DECIMAL(10,2));  
CREATE TABLE  
data_warehouse_utcv=# CREATE TABLE dim_tiempo ( id_tiempo SERIAL PRIMARY KEY, fecha DATE NOT N  
ULL, anio INT, mes INT, dia INT, trimestre INT);  
CREATE TABLE
```

```
data_warehouse_utcv=# CREATE TABLE fact_ventas ( id_venta SERIAL PRIMARY KEY, id_cliente INT REFERENC  
ES dim_cliente(id_cliente), id_producto INT REFERENCES dim_producto(id_producto), id_tiempo INT REFER  
ENCES dim_tiempo(id_tiempo), cantidad INT NOT NULL, monto_total DECIMAL(12,2) NOT NULL);  
CREATE TABLE
```

Carga de Datos

Simulación e importación de datos desde archivos CSV Para alimentar el Data Warehouse, se partió de la simulación de un entorno transaccional (OLTP) mediante la creación de conjuntos de datos de prueba almacenados en formato CSV (por ejemplo, clientes.csv, productos.csv y ventas.csv). Estos archivos representan las fuentes de información externas que contienen los registros diarios de operaciones del negocio, los cuales deben ser extraídos y procesados antes de su almacenamiento definitivo.

Normalización y carga al Data Warehouse usando Python El proceso de Extracción, Transformación y Carga (ETL) se llevó a cabo utilizando scripts desarrollados en Python ejecutados desde un entorno de Jupyter Notebook. Se empleó la librería **pandas** para la lectura de los archivos CSV, la limpieza de los datos (como el manejo de valores nulos y duplicados) y la normalización de los

mismos para asegurar que coincidieran exactamente con la estructura del modelo dimensional diseñado.

Una vez que los DataFrames estuvieron preparados y estandarizados, se utilizó la librería **SQLAlchemy** para establecer una conexión directa con el motor de base de datos PostgreSQL. A través de esta conexión, los registros fueron insertados de manera automatizada en sus respectivas tablas de dimensión y en la tabla de hechos central, garantizando en todo momento la integridad referencial de la información.

Simulación de Datos:

```
# 2. Simulación de datos para todas las tablas
datos_clientes = {
    'id_cliente': [1, 2, 3],
    'nombre': ['Alejandro', 'Perla', 'Alex'],
    'segmento': ['Estudiante', 'Profesional', 'Estudiante'],
    'ciudad': ['Córdoba', 'Veracruz', 'Córdoba'],
    'pais': ['México', 'México', 'México']
}

datos_productos = {
    'id_producto': [1, 2, 3],
    'nombre_producto': ['SSD Kingston NV3', 'Samsung Galaxy S26', 'Audífonos Bluetooth'],
    'categoria': ['Componentes', 'Telefonía', 'Accesorios'],
    'precio_unitario': [1100.50, 18500.00, 450.00]
}

datos_tiempo = {
    'id_tiempo': [1, 2, 3],
    'fecha': ['2026-05-05', '2026-05-07', '2026-06-10'],
    'anio': [2026, 2026, 2026],
    'mes': [5, 5, 6],
    'dia': [5, 7, 10],
    'trimestre': [2, 2, 2]
}

datos_ventas = {
    'id_venta': [1, 2, 3],
    'id_cliente': [1, 2, 3],
    'id_producto': [1, 2, 3],
    'id_tiempo': [1, 2, 3],
    'cantidad': [2, 1, 3],
    'monto_total': [2201.00, 18500.00, 1350.00]
}
```

Vigencia: 2020.09.17-2024.09.17
Registro: RPIL: 096

Dirección
Av. Universidad No. 350, Carretera Federal Cuitláhuac- La Tinaja
Congregación Dos Caminos, C.P. 94910 Cuitláhuac, Veracruz
Tel. (278) 73 2 20 50
www.utcv.edu.mx

Normalización de los datos y carga al Data Warehouse usando scripts en Python (pandas, SQLAlchemy).

```
# 4. Normalización y limpieza básica (Pandas)
df_clientes.fillna('Desconocido', inplace=True)
df_productos.fillna('Sin Categoría', inplace=True)
```

```
# 5. Carga al Data Warehouse
# Usamos una lista para garantizar el orden de carga
tablas_a_cargar = [
    ('dim_cliente', df_clientes),
    ('dim_producto', df_productos),
    ('dim_tiempo', df_tiempo),
    ('fact_ventas', df_ventas) # Siempre al final
]

print("Iniciando proceso ETL...")

for nombre_tabla, df in tablas_a_cargar:
    try:
        df.to_sql(nombre_tabla, engine, if_exists='append', index=False)
        print(f"✅ Datos cargados exitosamente en: {nombre_tabla}")
    except Exception as e:
        print(f"❌ Error al cargar {nombre_tabla}: {e}")

print("Proceso finalizado.")
```

```
Iniciando proceso ETL...
✅ Datos cargados exitosamente en: dim_cliente
✅ Datos cargados exitosamente en: dim_producto
✅ Datos cargados exitosamente en: dim_tiempo
✅ Datos cargados exitosamente en: fact_ventas
Proceso finalizado.
```

TABLA dim_cliente:

```
data_warehouse_utcv=# SELECT *FROM dim_cliente;
id_cliente | nombre | segmento | ciudad | pais
-----+-----+-----+-----+-----
1 | Alejandro | Estudiante | Córdoba | México
2 | Perla | Profesional | Veracruz | México
3 | Alex | Estudiante | Córdoba | México
(3 filas)
```

Vigencia: 2020/08/17-2024/08/17
Registro: RPH/L: 096

TABLA dim_producto:

```
data_warehouse_utcv=# SELECT *FROM dim_producto;
id_producto | nombre_producto | categoria | precio_unitario
-----+-----+-----+-----
          1 | SSD Kingston NV3 | Componentes |          1100.50
          2 | Samsung Galaxy S26 | TelefonYá |         18500.00
          3 | AudYfonos Bluetooth | Accesorios |           450.00
(3 filas)
```

TABLA dim_tiempo:

```
data_warehouse_utcv=# SELECT *FROM dim_tiempo;
id_tiempo | fecha | anio | mes | dia | trimestre
-----+-----+-----+-----+-----+-----
          1 | 2026-05-05 | 2026 | 5 | 5 | 2
          2 | 2026-05-07 | 2026 | 5 | 7 | 2
          3 | 2026-06-10 | 2026 | 6 | 10 | 2
(3 filas)
```

TABLA fact_ventas:

```
data_warehouse_utcv=# SELECT *FROM fact_ventas;
id_venta | id_cliente | id_producto | id_tiempo | cantidad | monto_total
-----+-----+-----+-----+-----+-----
          1 |          1 |          1 |          1 |          2 |         2201.00
          2 |          2 |          2 |          2 |          1 |         18500.00
          3 |          3 |          3 |          3 |          3 |         1350.00
(3 filas)
```

Análisis y Visualización

Uso de Jupyter Notebook para consultas SQL y procesamiento con pandas

Una vez que el Data Warehouse centralizó la información histórica y transaccional, la fase de explotación de datos se ejecutó en un entorno interactivo utilizando Jupyter Notebook. Para esto, se estableció una conexión de lectura hacia el motor PostgreSQL, lo que permitió ejecutar consultas estructuradas en lenguaje SQL directamente desde las celdas de código.

El flujo de trabajo consistió en utilizar sentencias JOIN para relacionar la tabla de hechos (ventas) con sus respectivas dimensiones (clientes, productos y tiempo), extrayendo indicadores clave de rendimiento. El resultado de estas consultas se importó a estructuras de tipo DataFrame utilizando la librería **pandas**. Esta integración metodológica es altamente eficiente, ya que delega el trabajo pesado de filtrado y cruce de información al motor de base de datos relacional, mientras aprovecha la flexibilidad de pandas para la inspección en

memoria, la manipulación estadística y la preparación final de los datos para su visualización.

Evidencia:

```
# 2. Consulta SQL: Uniendo Hechos y Dimensiones (Star Schema)
# Esta consulta trae el nombre del cliente, el producto, la fecha, y los totales.
consulta_sql = """
SELECT
    c.nombre AS cliente,
    c.ciudad,
    p.nombre_producto AS producto,
    p.categoria,
    t.fecha,
    t.mes,
    v.cantidad,
    v.monto_total
FROM
    fact_ventas v
INNER JOIN dim_cliente c ON v.id_cliente = c.id_cliente
INNER JOIN dim_producto p ON v.id_producto = p.id_producto
INNER JOIN dim_tiempo t ON v.id_tiempo = t.id_tiempo;
"""

print("Ejecutando consulta SQL en el Data Warehouse...")
```

Ejecutando consulta SQL en el Data Warehouse...

```
# 3. Procesamiento con pandas: Ejecutar consulta y guardar en DataFrame
try:
    df_analisis = pd.read_sql_query(consulta_sql, engine)
    print("✅ Datos extraídos correctamente.\n")

    # Mostramos los primeros registros para verificar
    print("--- Vista previa de los datos extraídos ---")
    display(df_analisis) # 'display' formatea bonito el DataFrame en Jupyter

    # 4. Ejemplo de procesamiento con pandas: Agrupación rápida
    print("\n--- Ventas Totales por Categoría de Producto ---")
    ventas_por_categoria = df_analisis.groupby('categoria')['monto_total'].sum().reset_index()
    display(ventas_por_categoria)

except Exception as e:
    print(f"❌ Error al ejecutar la consulta: {e}")
```

✅ Datos extraídos correctamente.

--- Vista previa de los datos extraídos ---

	cliente	ciudad	producto	categoria	fecha	mes	cantidad	monto_total
0	Alejandro	Córdoba	SSD Kingston NV3	Componentes	2026-05-05	5	2	2201.0
1	Perla	Veracruz	Samsung Galaxy S26	Telefonía	2026-05-07	5	1	18500.0
2	Alex	Córdoba	Audífonos Bluetooth	Accesorios	2026-06-10	6	3	1350.0

--- Ventas Totales por Categoría de Producto ---

	categoria	monto_total
0	Accesorios	1350.0
1	Componentes	2201.0
2	Telefonía	18500.0

Vigencia: 2020.09.17-2024.09.17
Registro: RPHL 096

Dirección
Av. Universidad No. 350, Carretera Federal Cuitláhuac- La Tinaja
Congregación Dos Caminos, C.P. 94910 Cuitláhuac, Veracruz
Tel. (278) 73 2 20 50
www.utcv.edu.mx

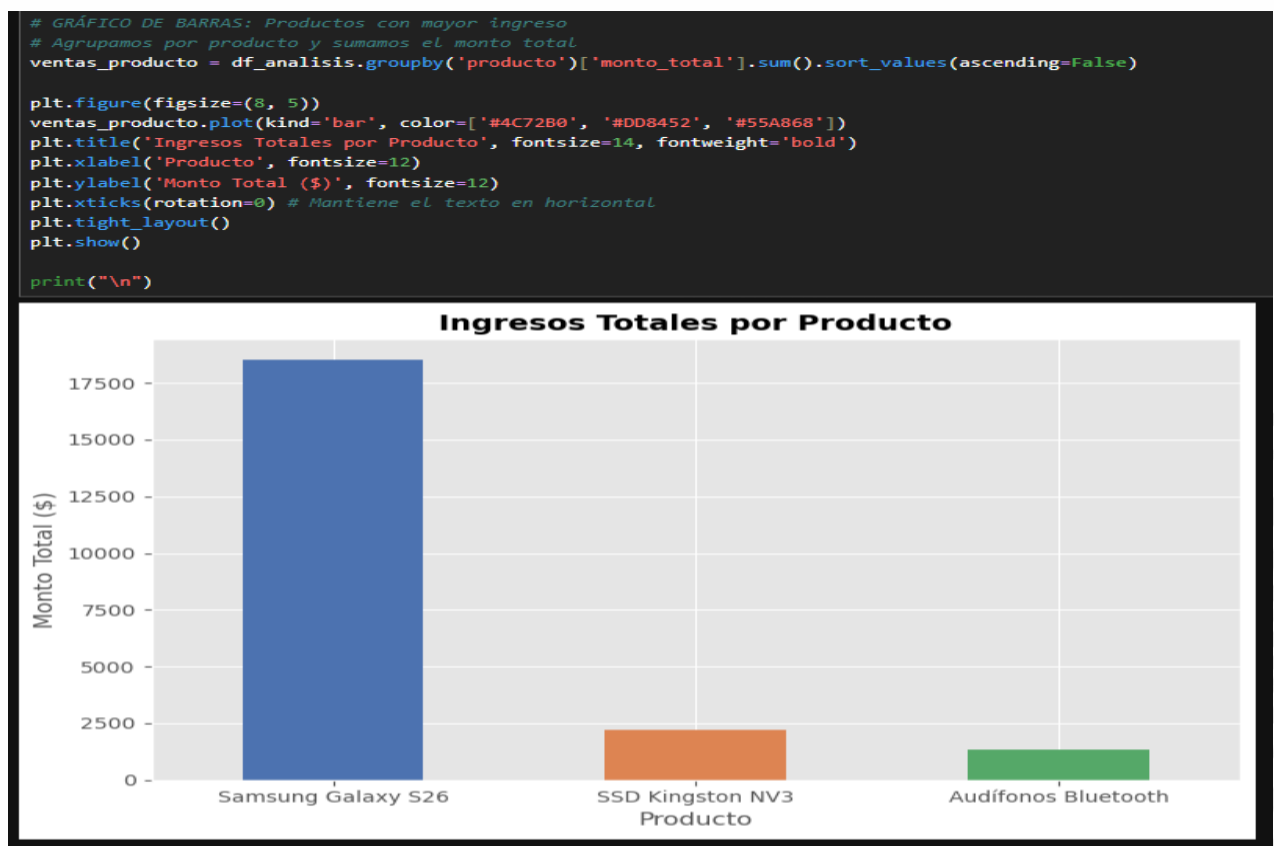
Generación de Gráficos e Indicadores Clave

Representación visual con Matplotlib El paso final en la explotación del Data Warehouse consistió en la generación de visualizaciones de datos para facilitar la interpretación de los indicadores clave de rendimiento (KPIs). Utilizando la librería Matplotlib en conjunto con los DataFrames previamente procesados por pandas, se construyeron representaciones gráficas que traducen los volúmenes de datos en información estratégica digerible.

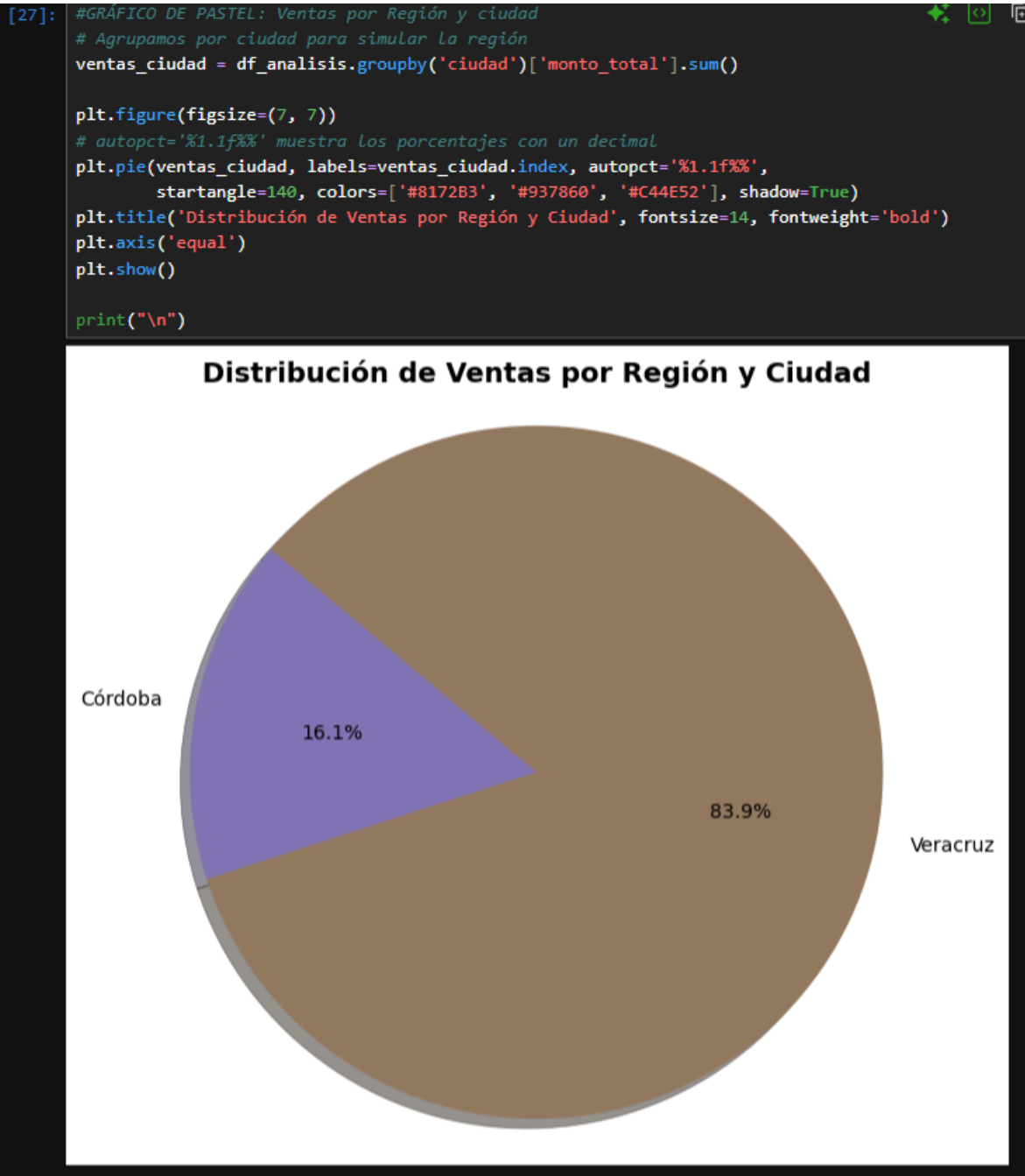
Se implementaron distintos tipos de gráficos según la naturaleza del indicador:

- Gráficos de barras: Utilizados para comparar magnitudes de forma categórica, permitiendo identificar rápidamente los productos más vendidos y los que generan mayores ingresos.
- Gráficos de pastel (pie charts): Empleados para mostrar la distribución porcentual de las ventas por región geográfica, evidenciando el peso y la contribución de cada zona al total de ingresos.
- Gráficos de líneas: Aplicados para trazar el comportamiento y la tendencia de las ventas a lo largo del tiempo, facilitando la detección de patrones estacionales basados en la dimensión de tiempo del modelo.

Grafica de Barras:



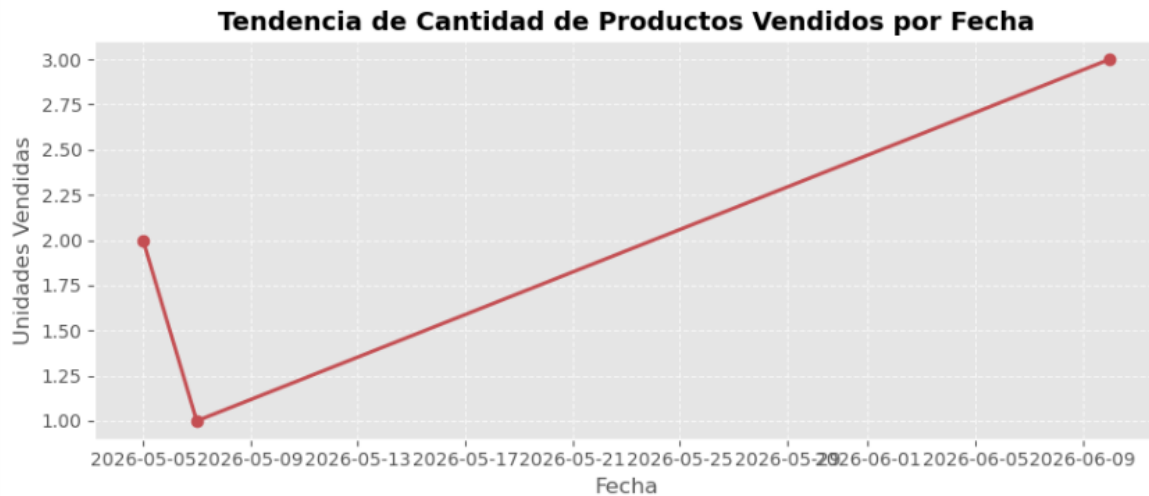
Grafica de Pastel:



Grafica de líneas:

```
#GRÁFICO DE LÍNEAS: Tendencia de ventas en el tiempo
# Agrupamos por fecha
ventas_tiempo = df_analisis.groupby('fecha')['cantidad'].sum()

plt.figure(figsize=(9, 4))
plt.plot(ventas_tiempo.index, ventas_tiempo.values, marker='o', linestyle='-', color='#C44E52', linewidth=2)
plt.title('Tendencia de Cantidad de Productos Vendidos por Fecha', fontsize=14, fontweight='bold')
plt.xlabel('Fecha', fontsize=12)
plt.ylabel('Unidades Vendidas', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```



Conclusiones

Para concluir, este proyecto me dejó muy claro que un Data Warehouse no es solo un concepto teórico, sino una herramienta súper necesaria en el mundo real. Cuando desarrollamos software y las plataformas empiezan a escalar, es muy fácil que la base de datos operativa colapse si intentamos sacar reportes pesados o analíticas directamente de ahí.

La mayor utilidad que le veo a un Data Warehouse es la ventaja de tener un entorno separado y optimizado puramente para el análisis. Nos permite agarrar un montón de datos históricos de diferentes fuentes, limpiarlos y convertirlos en gráficos que realmente ayudan a ver qué está pasando con el proyecto, todo sin afectar el rendimiento del sistema en el día a día. Al final, los datos no sirven de mucho si no podemos extraer conocimiento de ellos de forma rápida y eficiente para tomar mejores decisiones.